ThreatLens AI — Code Walkthrough

Technical Code Architecture & Line-by-Line Viva Defense Guide

Subject: Deep Learning (DS306)	**Course Instructor:** Dr. Kongara Srinivasa Rao
Submitted By: T Yokshith (23STUCHH010958)	**Submitted By:** Nitin Sain (23STUCHH010968)
Project Root: D:\yokshi dl\	**Framework:** PyTorch 2.11 + Streamlit 1.66

1. Project Architecture & File Directory

ThreatLens AI is an end-to-end Explainable Deep Learning Intrusion Detection System (IDS). It replaces black-box classifiers with a transparent Multi-Layer Perceptron (MLP) featuring first-layer weight-magnitude saliency for root-cause diagnosis. Below is the structural hierarchy of the codebase:

```
D:\yokshi dl\
■■■ src/
■   ■■■ model.py               # ThreatLensMLP PyTorch architecture & XAI explain_flow()
■   ■■■ train_model.py         # BCEWithLogitsLoss, Adam, ReduceLROnPlateau training loop
■   ■■■ data_preprocessing.py  # Zero-leakage StandardScaler, NaN cleaning, 70/15/15 split
■   ■■■ predict.py             # Inference pipeline & batch classification engine
■   ■■■ evaluate.py            # Confusion matrix, ROC-AUC, PR-AUC & latency metrics
■■■ pages/
■   ■■■ 1_Overview.py          # SOC telemetry, donut class distribution, alerts feed
■   ■■■ 2_Traffic_Analysis.py  # Real-time CSV ingestion, threshold slider, XAI gauge
■   ■■■ 3_Model_Evaluation.py  # Model vs baseline comparison & academic Viva Q&A;
■■■ app.py                    # Main Streamlit hub entrypoint
■■■ models/                   # Serialized PyTorch weights (.pt) & preprocessing (.pkl)
```

2. Core Deep Learning Model — src/model.py

This file defines the **ThreatLensMLP** neural network class and the **explain_flow()** feature attribution engine. It is the most critical file for your Deep Learning evaluation.

A. ThreatLensMLP Architecture Breakdown

The model is a deep feed-forward Multi-Layer Perceptron implemented via torch.nn.Sequential:

```
class ThreatLensMLP(nn.Module):
    def __init__(self, input_dim: int, hidden_dim1: int = 64, hidden_dim2: int = 32, dropout_rat
e: float = 0.2):
        super(ThreatLensMLP, self).__init__()
        self.network = nn.Sequential(
            # Layer 1: 59 inputs -> 64 hidden units
            nn.Linear(input_dim, hidden_dim1),
            nn.BatchNorm1d(hidden_dim1),
            nn.ReLU(),
            nn.Dropout(dropout_rate),

            # Layer 2: 64 -> 32 hidden units
            nn.Linear(hidden_dim1, hidden_dim2),
            nn.BatchNorm1d(hidden_dim2),
            nn.ReLU(),
            nn.Dropout(dropout_rate),

            # Layer 3: 32 -> 1 output logit (unbounded scalar)
            nn.Linear(hidden_dim2, 1)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.network(x)

    def predict_proba(self, x: torch.Tensor) -> torch.Tensor:
        self.eval()
        with torch.no_grad():
            return torch.sigmoid(self.forward(x))
```

Key Architectural Concepts to Explain to Dr. Kongara:

- **Input Dimension (59 Features):** Derived after removing zero-variance and redundant identifier columns from 78 raw CIC-IDS2017 features.
- **BatchNorm1d:** Normalizes batch activations to mean=0, var=1. This accelerates convergence and prevents internal covariate shift.
- **Dropout (p=0.2):** Zeroes out 20% of neuron outputs randomly during training to prevent co-adaptation and overfitting.
- **Raw Logits vs Sigmoid:** The output layer returns raw unnormalized logits so we can use numerically stable BCEWithLogitsLoss during training, while applying torch.sigmoid only at test time.

B. Explainable AI Engine — explain_flow()

Instead of opaque predictions, ThreatLens computes **first-layer weight-magnitude saliency**. It measures how much each feature is amplified by the first dense layer multiplied by the actual standardized input deviation:

```
def explain_flow(self, x_single: torch.Tensor, feature_names: List[str], top_k: int = 5):
    self.eval()
    with torch.no_grad():
        x_val = x_single.squeeze(0) if x_single.ndim > 1 else x_single
        W1 = self.network[0].weight             # First Linear layer [64, 59]
        mean_abs_weight = W1.abs().mean(dim=0)  # Sensitivity per input feature
        saliency = (mean_abs_weight * x_val.abs()).cpu().numpy()

    top_indices = np.argsort(saliency)[::-1][:top_k]
    return [{'feature': feature_names[i], 'attribution_score': float(saliency[i])} for i in top_
indices]
```

3. Training Pipeline & Loss Function — src/train_model.py

This script orchestrates the model optimization, handling class imbalance, dynamic learning rates, and early stopping. It also trains a baseline Logistic Regression model for academic comparison.

```
# 1. Class Imbalance Reweighting
pos_weight = torch.tensor([3.0]).to(device)  # Penalizes missed intrusions 3x heavier
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# 2. Optimizer & Dynamic Scheduler
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=3)

# 3. Epoch Loop with Early Stopping
for epoch in range(1, epochs + 1):
    model.train()
    optimizer.zero_grad()
    logits = model(x_train_batch).squeeze(-1)
    loss = criterion(logits, y_train_batch)
    loss.backward()
    optimizer.step()

    # Validation & Early Stopping Check
    scheduler.step(val_loss)
    if val_f1 > best_val_f1:
        torch.save(model.state_dict(), 'models/mlp_model.pt')
```

Key Training Concepts to Defend:

- **pos_weight=3.0:** In cybersecurity, a False Negative (undetected malware) is far more dangerous than a False Positive. The 3.0 weight forces the loss function to prioritize catching intrusions.
- **ReduceLROnPlateau:** When validation loss plateaus for 3 epochs, the learning rate drops by 50% (factor=0.5), allowing fine-grained convergence into local minima.
- **Baseline Comparison:** We train a `LogisticRegression(class_weight='balanced')` on the exact same splits to prove the non-linear MLP's superiority.

4. Preprocessing & Leakage Prevention — src/data_preprocessing.py

Rigorous data hygiene is required for academic integrity. This module cleans real-world network captures and isolates train/val/test splits before computing normalization parameters.

```
def preprocess_cicids_pipeline(df_raw, test_size=0.15, val_size=0.15, random_state=42):
    # Step 1: Clean column names (strip leading/trailing whitespace)
    df = clean_column_names(df_raw)

    # Step 2: Handle infinities and NaNs (replace with 0 or column max)
    df = clean_features(df)

    # Step 3: Stratified Split BEFORE scaling (Prevents Data Leakage!)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, stratify=y)
    X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, stratify=y_t
emp)

    # Step 4: Fit StandardScaler ONLY on X_train
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)      # Transform only
    X_test_scaled = scaler.transform(X_test)    # Transform only

    # Save scaler and feature names to models/preprocessing.pkl
```

Zero Data Leakage Rule:

If a model fits StandardScaler on the entire dataset before splitting, the mean and variance of the test data leak into training. In ThreatLens AI, scaler.fit() is strictly restricted to X_train, guaranteeing completely blind, honest test split evaluation.

5. Inference & Evaluation — src/predict.py & src/evaluate.py

ThreatLensPredictor provides an enterprise-ready inference class. It categorizes incoming flows into four operational risk levels based on model confidence:

BENIGN	0.00 to 0.35	Normal authorized traffic. No action required.
ELEVATED	0.35 to 0.50	Mild anomaly (traffic spikes / unusual port). Logged.
SUSPICIOUS	0.50 to 0.70	Flagged attack candidate. Queue for analyst review.
CRITICAL	0.70 to 1.00	High-confidence intrusion. Instant firewall block alert.

6. Academic Viva Defense Cheat Sheet (Questions for Sir)

Dr. Kongara Srinivasa Rao is likely to ask technical questions about why specific architectural and mathematical choices were made. Use these prepared answers:

Q1: Why did you use Multi-Layer Perceptron (MLP) instead of CNN or LSTM?

A: Network flow data from CIC-IDS2017 is aggregated tabular data (features like Flow Duration, Total Packets, IAT), not raw byte sequences or images. MLPs with non-linear activations excel at tabular feature correlations and achieve sub-millisecond (0.51 ms) execution, making them ideal for high-speed network gateways.

Q2: Why use BCEWithLogitsLoss instead of BCELoss + Sigmoid in the forward pass?

A: Combining the Sigmoid activation and Binary Cross Entropy into a single layer (BCEWithLogitsLoss) takes advantage of the log-sum-exp trick. This prevents severe numerical instability, floating-point overflow, and

exploding/vanishing gradients during backpropagation.

Q3: How does your Explainable AI (XAI) work mathematically?

A: We compute first-layer weight-magnitude saliency: Saliency_j = Mean(|W1[:, j]|) * |x_j|. It multiplies the neural network's structural sensitivity to feature j (the weights connecting feature j to layer 1) by the standardized deviation of the input. This highlights features that pushed the score highest without needing slow autograd backward passes in production.

Q4: How did you ensure your model does not overfit?

A: We used three distinct regularization techniques: (1) Dropout layers (p=0.2) after every hidden activation, (2) BatchNorm1d to regularize activations across batches, and (3) Weight decay (L2 regularization = 1e-4) in the Adam optimizer.

Q5: What happened when the Anomaly Score Threshold slider was changed in the app?

A: The threshold slider acts as an operational noise filter. At 0.00, analysts see all traffic. At 0.25, the dashboard filters out the 809 normal benign flows (score 0.00), isolating only suspicious flows and critical alarms for triage.